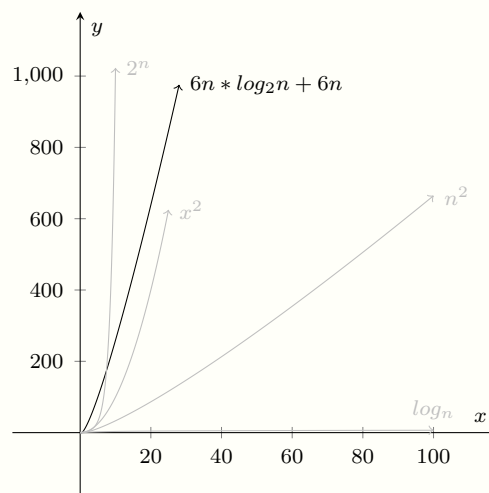


Merge Sort

1 Description

Recursive algorithm based on the divide and conquer strategy. It is based on two components: the function that split the given list recursively until it receives a single item list, and a merge function that is used on the recursive algorithm to merge two already sorted lists into a combined sorted one.

2 Time Complexity



3 Pseudocode

Input: $A = [A_1, A_2, \dots, A_n]$
Output: $B = [B_1, B_2, \dots, B_n]$

```
function MERGESORT(A)                                ▷ Algorithm itself
     $n \leftarrow \text{length}(A)$ ;
    if  $n \leq 1$  then:
        return A;                                     ▷ Length 1, already sorted, return
    else
         $l \leftarrow [A_1, \dots, A_{n/2}]$ ;             ▷ Split left subarray
         $r \leftarrow [A_{(n/2)+1}, \dots, A_n]$ ;         ▷ Split right subarray
         $L \leftarrow \text{MERGESORT}(l)$ ;                  ▷ Sort recursively
         $R \leftarrow \text{MERGESORT}(r)$ ;                  ▷ Sort recursively
         $B \leftarrow \text{MERGE}(L, R)$ ;                  ▷ Merge both subarrays
    return B;
end if
```

end function

function MERGE(L, R)

$n \leftarrow \text{length}(L) + \text{length}(R);$

▷ Full length of resulting array

$B \leftarrow [\];$

▷ Initialize empty array

$i \leftarrow 0;$

$j \leftarrow 0;$

for $k \leftarrow 0$ to n **do**

▷ Iterate full length of both subarrays

if $i \leq \text{length}(L)$ **and** $j \leq \text{length}(R)$ **then**

▷ Both within range

if $L[i] \leq R[j]$ **then:**

▷ Left lower, set it

$B[k] \leftarrow L[i];$

$i \leftarrow i + 1;$

else if $L[i] > R[j]$ **then:**

▷ Right lower, set it

$B[k] \leftarrow R[j];$

$j \leftarrow j + 1;$

end if

else if $i + 1 \leq \text{length}(L)$ **then:**

▷ Only left within range, set it

$B[k] \leftarrow L[i];$

$i \leftarrow i + 1;$

else if $j + 1 \leq \text{length}(R)$ **then:**

▷ Only right within range, set it

$B[k] \leftarrow R[j];$

$j \leftarrow j + 1;$

end if

end for

return $B;$

end function